

Docker

Docker – Breve Introduzione e Installazione

Docker è un tool che **semplifica il ciclo di vita del software**. Prima di lui, per avere ambienti riproducibili si usavano le **virtual machine**, che però erano troppo pesanti: ogni VM sullo stesso computer portava con sé **un intero sistema operativo**, quindi per sviluppare serviva una macchina molto potente. Docker ha un funzionamento simile per scopo ma non installa un sistema operativo per ogni macchina: si basa sui **container**, cioè l'insieme dei dati di cui un'applicazione ha bisogno per essere eseguita — librerie, file system, file di configurazione, script. Il docente dice che è diventato uno **standard di fatto**.

I due concetti chiave: immagine e container. È la metafora che ritorna in tutto il corso:

L'**immagine** è come il **negativo** di una macchina fotografica, il **container** è la **foto** vera e propria.

E, siccome siamo in Java, la seconda metafora (più precisa):

Docker	Java
immagine	la classe (contiene metodi, campi... tutte le informazioni)
container	l' istanza/oggetto (l'avvio reale di quella classe)

Da **una sola immagine** puoi istanziare **quanti container vuoi**. Esempio del docente: ti serve MySQL? Scarichi l'immagine MySQL e lanci il container. Oppure crei tu l'immagine della tua applicazione Spring Boot e da quella lanci più container.

Installazione. Si cerca "docker download windows", si scarica **Docker Desktop**, che si installa da solo. Poi lo si esegue (come amministratore, oppure lasciandolo avviare in automatico con il sistema operativo). Dentro Docker Desktop si vedono **tutte le immagini** e **tutti i container** presenti sulla macchina.

Il registry. Il docente si è registrato su Docker Hub creando un profilo: sotto quel profilo vengono memorizzate le sue immagini sul **repository ufficiale di Docker**. Precisazione importante: ogni azienda ha in genere **i propri repository**, e quando lavori usi quelli aziendali, non l'hub pubblico.

Nel video successivo userà l'immagine `hello-world`, che scarica l'immagine e avvia un container che non fa altro che stampare la scritta "Hello World".

Virtual Machine vs Container

Cos'è una VM. Una macchina virtuale è un **computer dentro il tuo computer**: ha un proprio sistema operativo (Windows, Mac, Linux) eseguito sopra il tuo sistema operativo fisico. Puoi averne più di una, ognuna con il suo SO, e dentro ognuna installi librerie, dipendenze, tool e ci sposti il codice sorgente. Risolvi quindi il **problema della portabilità**.

I problemi delle VM (elencati dal docente):

- **Enorme sovraccarico** — ogni VM è quasi davvero un computer autonomo in esecuzione: occupa RAM e spazio. Con più VM sovraccarichi la macchina fisica.
- **Duplicazione di dati ridondanti** — il pezzo duplicato più grosso è proprio il **sistema operativo**, che invece vorresti condiviso. Da qui **spreco di risorse** e prestazioni negative.
- **Configurazione complicata** — anche se riproducibile e condivisibile, configurare una VM su ogni sistema è laborioso: **non esiste un unico file di configurazione**. Se devo portare la mia applicazione da sviluppo a produzione, devo assicurarmi che la macchina di produzione sia identica alla virtuale, e non è immediato.

Come i container risolvono.

- Sono **molto più leggeri**: **non contengono un sistema operativo**, si appoggiano al SO della macchina ospite. Il SO è "gonfio", con tonnellate di strumenti extra che ti rallentano.
- Contengono **solo gli strumenti di cui hai bisogno**, niente tool inutilizzati.
- Si configurano **con un semplice file di configurazione condivisibile**: passi il file a chiunque e lui **ricrea lo stesso identico contenitore** in automatico. (È l'anticipazione di Dockerfile e docker-compose.)

Nel mezzo c'è il **container engine** (il docente lo chiama "content engine"), il motore che si occupa di eseguire i vari container.

In sintesi: VM e container fanno concettualmente la stessa cosa, ma il container è **più veloce**, **occupa meno spazio** e **si configura/condivide molto più facilmente**, perché non si porta dietro un intero sistema operativo.

Lo schema mentale da avere:



Docker – Concetti Base

```
docker run hello-world
```

Nota: il docente sbaglia due volte — prima scrive `hello world` staccato (errore), poi si accorge che **Docker Desktop non era avviato** (senza il motore acceso, nessun comando funziona). Corretto il tutto, l'output racconta esattamente cosa succede:

"Unable to find image locally" → la scarica → poi avvia il container che stampa il messaggio.

Il meccanismo fondamentale. Quando chiedi a Docker di eseguire un container, lui:

1. controlla se l'**immagine esiste in locale**;
2. se non c'è, la **scarica** da `hub.docker.com`;
3. **avvia il container**.

Cos'è un Docker Registry. `hub.docker.com` è un **registry**: contiene una serie di **repository**, ufficiali e non. Un **repository** è un *contenitore di immagini correlate tra loro*, suddivise per **tag** (le versioni: `latest`, `1.0`, `1.01`, `1.02...`). Se non specifichi il tag, **scarica automaticamente latest**.

Gerarchia da tenere a mente:

REGISTRY (`hub.docker.com`)

└─ REPOSITORY (es. `hello-world`, `mysql`, `programmingacademydotit/docker-springboot-helloworld-2`)

└─ IMAGE:TAG (`:latest`, `:0.0.1-SNAPSHOT`, `:0.0.2-SNAPSHOT ...`)

└─ CONTAINER, CONTAINER, CONTAINER ... ← tutti dalla stessa immagine

Vedere cosa hai in locale:

`docker images`

Secondo esempio: un'immagine con un servizio REST. Il docente alza l'asticella e usa un'immagine creata da lui, pubblica sull'hub:

`docker run programmingacademydotit/docker-springboot-helloworld-2:0.0.1-SNAPSHOT`

Docker scarica l'immagine (il primo download richiede tempo — ecco perché prima controlla sempre se ce l'hai già) e **Spring Boot parte da solo**: nei log compare l'avvio sulla porta 8080. L'applicazione è esattamente quella che trovi nello zip:

@RestController

```
public class HelloController {
```

```
    @GetMapping("esempio")
```

```
    public String esempio() {
```

```
        return "hello2";
```

```
    }
```

```
}
```

Il problema delle reti (il punto più importante della lezione). Il docente prova `localhost:8080/esempio` e riceve **errore**, pur vedendo Spring avviato correttamente. Va persino a ricontrollare l'URL su Eclipse: l'URL era giusto. Il motivo vero è:

Docker crea una sua rete interna tra i container, separata dalla nostra.

Bisogna quindi **mappare le porte** con l'opzione `-p`, che va messa **prima del nome dell'immagine**:

```
docker run -p 8080:8080 programmingacademydotit/docker-springboot-helloworld-2:0.0.1-SNAPSHOT
```

Traduzione: *"il container è in ascolto sulla 8080; collegala alla nostra rete sulla 8080"*. Stavolta `localhost:8080/esempio` risponde **hello2**. E il secondo avvio è immediato perché l'immagine non viene più scaricata.

Dimostrazione che le due reti sono davvero separate. Rilancia mappando porte diverse:

```
docker run -p 8081:8080 <immagine>
```

Il container resta in ascolto sulla **8080 internamente** (come da `application.properties` dell'immagine), ma **fuori** lo raggiungi solo su `localhost:8081`. Infatti `localhost:8080` non risponde più, `localhost:8081` sì. La sintassi è quindi: **-p portaEsterna:portaInterna**.

Detach: lasciare il container in esecuzione. Se ti stacchi dal prompt, il container viene stoppato. Con `-d` lui gira in background:

```
docker run -d -p 8083:8080 <immagine>
```

Perdi i log a video (non sei più "in tail"), ma il servizio continua a rispondere: ha un'**esecuzione indipendente dal prompt**.

Vedere i container e i log.

```
docker container ls      # elenco container avviati (serve LS, non basta "docker container")
```

```
docker logs <ID_CONTAINER> # attenzione: logs, non log
```

```
docker logs -f <ID_CONTAINER> # -f = tail, log in tempo reale
```

Con `-f` attivo, se invochi il servizio vedi i log comparire live — è il `tail` che si usa anche su Windows.

Più container dalla stessa immagine. Basta **cambiare la porta esterna** (8081, 8083...): `docker container ls` ne mostra due, entrambi della stessa immagine. È la dimostrazione pratica del rapporto classe → istanze.

Stoppare:

```
docker container stop <ID_CONTAINER>
```

Dopo lo stop, `docker container ls` non mostra più nulla.

Docker – Comandi Principali

Lezione di rassegna dei comandi. (Anche qui parte con un errore perché Docker Desktop non era avviato — il **Docker engine** deve girare.)

Immagini.

```
docker images          # repository + tag presenti in locale
docker pull mysql      # senza tag → scarica automaticamente :latest
docker image rm <ID_IMMAGINE> # rimuove un'immagine
```

Immagini ufficiali. Concetto spiegato qui: **Docker paga una serie di sviluppatori per mantenere i repository più utilizzati** (come MySQL). Quelli forniti da Docker sono le **immagini ufficiali**.

Container.

```
docker container ls      # elenco container
docker container pause <ID> # mette in pausa
docker container unpause <ID> # riprende
docker container stop <ID>  # stop "gentile"
docker container kill <ID>  # forza la terminazione
```

Dettaglio su **pause**: il container **rimane attivo** (nei log lo vedi vivo), **ma se provi a invocarlo non ottieni nessuna risposta**. **kill** serve quando un container rimane sospeso e non riesci a bloccarlo.

Digitando solo `docker container` compare l'**elenco dei comandi possibili** — utile per esplorare. Per la lista completa il docente rimanda alla **documentazione ufficiale su Docker Docs**, perché i comandi sono tantissimi.

Da GUI. Le stesse informazioni base si trovano in **Docker Desktop**: sezione immagini (locali e, se sei loggato, quelle sull'hub) e sezione container, con la possibilità di filtrare per mostrare solo quelli avviati.

Docker – Creazione Immagine Locale e Remota

Obiettivo. Creare l'immagine **della propria applicazione Spring Boot** e pubblicarla sull'hub. Punto interessante: **non si scrive nessun Dockerfile** — infatti nello zip non c'è. Si usa il **Spring Boot Maven Plugin**.

La configurazione nel pom.xml (questa è reale, presa dal progetto dello zip):

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <image>
          <name>programmingacademydotit/${project.artifactId}:${project.version}</name>
```

```

        </image>

        <pullPolicy>IF_NOT_PRESENT</pullPolicy>

    </configuration>

</plugin>

</plugins>

</build>

```

Come si legge: "crea un'immagine chiamata *programmingacademydotit/* + l'**artifactId** del progetto, con come tag la **versione** del progetto". Nel nostro `pom.xml` `artifactId` è `docker-springboot-helloworld-2` e `version` è `0.0.3-SNAPSHOT` → l'immagine si chiamerà `programmingacademydotit/docker-springboot-helloworld-2:0.0.3-SNAPSHOT`. La `pullPolicy IF_NOT_PRESENT` dice di non ricaricare ciò che è già presente.

Nota sui tag. Il docente parte da un repository che ha già i tag `0.0.1` e `0.0.2-SNAPSHOT` e vuole creare il `0.0.3-SNAPSHOT`. Sottolinea che **creare un nuovo tag, una nuova immagine o un nuovo repository si fa allo stesso identico modo**: cambi la versione nel pom.

I due passi.

1. **Creare il jar** — `maven clean + maven install`. A fine build, in `target/` compare `docker-springboot-helloworld-2-0.0.3-SNAPSHOT.jar` (è nello zip, puoi verificarlo).
2. **Creare l'immagine** — in Eclipse, *Run Configurations* → *Maven Build* → *nuovo task*, con goal:

```
spring-boot:build-image -DskipTests
```

Due dettagli pratici che dà: nel campo *Base directory* conviene incollare la variabile del **progetto selezionato** (`${project_loc}`), così il task **non è legato a un singolo progetto** ma è lanciabile su qualsiasi progetto; e `-DskipTests` per non eseguire i test.

La **prima creazione dell'immagine impiega parecchio tempo**, poi diventa rapida. E fa tutto in automatico, "senza configurazioni complicatissime".

Verifica. `docker images` (o Docker Desktop → *Images* → Refresh) mostra la nuova `0.0.3-SNAPSHOT`. Ma nella scheda dell'**hub** la `0.0.3` **non c'è**: si vedono solo `0.0.1` e `0.0.2`. Quella appena creata è **solo locale**.

Pubblicazione in remoto:

```
docker push <nome-immagine>:<tag>
```

Dopo il push, aggiornando la pagina dell'hub compare il tag `0.0.3-SNAPSHOT`.

Docker Compose – Introduzione

Docker Compose, come dice la parola, è un "**compositore**" di container Docker: serve ad avviare **più container insieme**, cosa che succede sempre in un'applicazione a microservizi (dove hai i

servizi più tutti i tool di contorno: config, load balancer, Eureka, ecc.). È **già installato insieme a Docker Desktop**, non va scaricato a parte.

Si configura con **un file YAML**. E il docente insiste su un punto: **gli spazi sono importanti** (2 spazi di indentazione per il nome del servizio, 4 per le sue proprietà).

Il file reale (docker-compose.yml dello zip):

```
version: '3.7'

services:
  docker-compose-springboot-helloworld:
    image: programmingacademydotit/docker-springboot-helloworld-2:0.0.1-SNAPSHOT
    mem_limit: 700m
    ports:
      - "8081:8081"
    networks:
      - prova-network

networks:
  prova-network:
```

Riga per riga:

- `version: '3.7'` — versione della sintassi Compose;
- `services:` — sotto ci va **l'elenco di tutti i servizi da avviare** (qui uno solo, ma potresti metterne quanti vuoi);
- il nome del servizio (lo scegli tu), poi `image:` con **nome immagine + tag**;
- `mem_limit: 700m` — limite di memoria imposto al container;
- `ports:` — la mappatura "esterna:interna", come il `-p` di prima;
- `networks:` dentro il servizio → a **quale rete si collega**; `networks:` in fondo al file → **definizione della rete**.

Avvio. Dal prompt, **nella cartella in cui si trova il file YAML** (lo cerca lì dentro e lo legge):

```
docker-compose up
```

Le due sbucciature del docente (che sono la parte didattica migliore).

1. **Primo tentativo, senza networks:** il container parte ma `localhost:8080`/esempio non risponde. Stesso identico motivo della lezione 3: il container **sta in una rete separata dalla nostra**, quindi va configurata la network. Aggiunte le due sezioni `networks`, rilancia e nell'output compare *"Network prova-network Created"*.
2. **Secondo tentativo, ancora 404:** e qui scopre di aver mappato la porta sbagliata. L'immagine, nel suo `application.properties`, espone il servizio sulla **8081**, non

sulla 8080. Nel compose aveva scritto "8080:8080", dove **dalla parte interna non c'era nessuno in ascolto** → *not found*. Corretto in "8081:8081",
localhost:8081/esempio risponde **hello2**.

Avvertimento sulle versioni, ripetuto in tutte le lezioni successive: bisogna stare attentissimi a usare **versioni compatibili tra loro**, altrimenti il sistema non parte. Le versioni esatte del corso, che trovi anche scritte nel commento in fondo a `HelloController.java`:

Spring Boot 2.7.14-SNAPSHOT · Spring Cloud 2021.0.8 · Docker 24.0.2 · Java 17 · YAML version 3.7

Il consiglio metodologico (che ripeterà ogni volta): prima **scarica il suo file YAML e fallo funzionare con le sue immagini pubbliche**. Se funziona, vuol dire che il tuo ambiente è configurato bene. Solo dopo prova a rifare un esempio da zero con le tue immagini.

Docker Compose – Esempio con Eureka Server

Obiettivo. Aggiungere al compose, oltre alla propria applicazione, anche il **naming server (Eureka)** — e far sì che l'applicazione **si registri** su di esso.

Passo 1: preparare una nuova versione dell'immagine. Nel progetto, la dipendenza Eureka client e la property erano **commentate** (le vedi commentate anche nei file dello zip):

```
<!-- <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency> →
```

```
#eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka
```

Il docente **le decommenta**, alza la versione nel pom a 0.0.2-SNAPSHOT, fa `clean install` e poi lancia il task Maven creato nella lezione 5.

Novità: push automatico. Stavolta **non dà docker push**: l'immagine viene pubblicata da sola. Il motivo è che nel plugin ha aggiunto le credenziali del registry:

```
<image>
<name>programmingacademydotit/${project.artifactId}:${project.version}</name>
<publish>true</publish>
</image>
<docker>
<publishRegistry>
<username>*****</username>
```



```
<password>*****</password>
<email>*****</email>
</publishRegistry>
</docker>
```

(Nel video gli asterischi ci sono davvero, per non mostrare user e password.) Sull'hub compaiono ora **due tag**: 0.0.1-SNAPSHOT (senza Eureka) e 0.0.2-SNAPSHOT (con Eureka).

Passo 2: il compose. Questo è **docker-compose2.yml** dello zip, per intero:

```
version: '3.7'

services:

  docker-springboot-helloworld-2:
    image: programmingacademydotit/docker-springboot-helloworld-2:0.0.2-SNAPSHOT
    mem_limit: 700m
    ports:
      - "8081:8081"
    networks:
      - prova-network
    depends_on:
      - naming-server
    environment:
      EUREKA.CLIENT.SERVICEURL.DEFAULTZONE: http://naming-server:8761/eureka

  naming-server:
    image: programmingacademydotit/docker-compose-eureka-server-1:0.0.1-SNAPSHOT
    mem_limit: 700m
    ports:
      - "8761:8761"
    networks:
      - prova-network

networks:
```

prova-network:

Le due chiavi nuove, che sono il cuore della lezione:

- **depends_on** — dichiara che il mio servizio **dipende dal container naming-server**, quindi Compose avvia prima quello.
- **environment** — **sovrascrive una property** dell'applicazione tramite variabile d'ambiente. Serve perché nel `application.properties` dell'immagine l'indirizzo di Eureka è `http://localhost:8761/eureka`, e **dentro Docker localhost non va bene**.

Il concetto da portarsi a casa (il docente lo ripete tre volte, ed è la cosa che confonde tutti):

Dentro la rete Docker, **il nome del servizio scritto nel compose diventa l'hostname della macchina**. Quindi non ti colleghi a `localhost:8761` ma a **naming-server:8761**, dove `naming-server` è esattamente il nome che ho scelto qui sotto per il container di Eureka.

Attenzione quindi alla coerenza: lo stesso identico nome deve comparire in **tre punti** — come nome del servizio, dentro `depends_on`, e dentro l'URL della variabile d'ambiente.

Avvio e verifica.

`docker-compose up`

Compose crea la rete, poi il container `naming-server`, poi la nostra applicazione. Andando su **localhost:8761** si apre la console di Eureka e si vede **HELLO-WORLD-2 registrata correttamente**. (Ricorda che quel `localhost:8761` funziona perché il compose ha mappato "8761:8761".)

Docker Compose – Esempio con API Gateway

Obiettivo. Continuazione diretta del video precedente: all'architettura (applicazione + Eureka) si aggiunge il **gateway**.

La modifica al file è quasi un copia-incolla del blocco già visto:

```
api-gateway:
```

```
  image: programmingacademydotit/docker-compose-gateway:0.0.1-SNAPSHOT
```

```
  mem_limit: 700m
```

```
  ports:
```

```
    - "8765:8765"
```

```
  networks:
```

```
    - prova-network
```

```
  depends_on:
```

```
    - naming-server
```

environment:

EUREKA.CLIENT.SERVICEURL.DEFAULTZONE: <http://naming-server:8761/eureka>

La logica è identica: **anche il gateway è un client Eureka**, quindi anche lui deve registrarsi e anche a lui va sovrascritta la stessa variabile d'ambiente con l'indirizzo `naming-server:8761`. Le porte convenzionali della lezione restano **8761 per Eureka e 8765 per il gateway**.

L'intoppo didattico. Il primo `docker-compose up` fallisce con **"immagine non trovata"**: aveva scritto nel `compose` una versione dell'immagine che **non aveva ancora creato**. Rifà quindi la sequenza completa: `clean install` → `task Maven spring-boot:build-image` (che crea e pubblica sull'hub) → e solo allora `docker-compose up`.

Verifica. Sulla console di Eureka (`localhost:8761`) compaiono ora **due istanze registrate**: l'applicazione REST e il gateway.

Il consiglio finale è di nuovo lo stesso, con un'aggiunta netta: **Docker Compose è molto sensibile alle versioni** di Spring Boot / Spring Cloud / Docker. Fai girare prima il file del corso con le immagini pubbliche del docente, poi sostituisci le tue.

Zipkin – Introduzione e creazione tramite Docker

Il problema. In un'architettura a microservizi arriva la segnalazione: *"quando il servizio X chiama il servizio Y non mi risponde"*. Con centinaia o migliaia di microservizi, capire **dove** sia l'errore leggendo i log sparsi è un incubo.

Definizione. Zipkin è un server di **tracing distribuito**: traccia automaticamente tutte le chiamate tra servizi e le **correla tra loro**. Se il servizio 1 chiama il 2 che chiama il 3, Zipkin lega tutte queste chiamate a un identificativo comune.

Trace e Span (la coppia che il docente disegna):

- il **trace** è l'intera richiesta **end-to-end** — attraversa tutti i servizi coinvolti ed è lo stesso per tutti (`trace 1`);
- lo **span** è il **singolo salto** dentro quel percorso (`span 1`, `span 2`, `span 3`, `span 4`).

Quindi vedi la stessa richiesta come `trace1/span1`, `trace1/span2`, ecc.: tutto già mappato e tracciato, e Zipkin ti offre in più una **dashboard comodissima** per consultare queste tracce.

Avviare Zipkin con Docker — una riga sola, senza configurare niente:

```
docker run -p 9411:9411 openzipkin/zipkin:2.23
```

Docker non trova l'immagine in locale, la scarica e la avvia. Il server è pronto sulla porta **9411**.

Configurare l'applicazione. Due dipendenze (Spring Cloud Sleuth + l'esportatore verso Zipkin):

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-sleuth</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-sleuth-zipkin</artifactId>
</dependency>
```

E due property — nel video non ne legge i nomi, ma sono le due standard di Sleuth: quanto campionare le tracce e **dove ascolta il server Zipkin**:

```
spring.sleuth.sampler.probability=1.0
spring.zipkin.base-url=http://localhost:9411
```

L'esperimento. Un servizio REST (zipkin-application-rest) che **invoca un secondo servizio REST**, che è la nostra vecchia conoscenza `hello-world-2` (quella che ritorna `hello2`).

Anche qui inciampa in modo istruttivo: **entrambe le applicazioni erano configurate sulla 8081**, quindi la seconda non partiva — **conflitto di porta**. Sposta la prima sulla 8080 e la seconda resta sulla 8081. (Compare anche un errore di registrazione su Eureka, residuo dell'esempio precedente: **non influisce** sull'esperimento.)

Invoca `localhost:8080/esempio`, che dietro le quinte chiama la seconda applicazione, e riceve la risposta. Poi apre **localhost:9411**, la console di Zipkin, clicca **Run** e vede tutte le chiamate tracciate, navigabili per trace.

Perché conta. Zipkin è utile in sviluppo, ma **soprattutto in produzione**: quando c'è un errore, capire quale microservizio l'ha generato senza tracing è, testualmente, un bel casino.

Docker Compose – Esempio con Eureka Server, API Gateway e Zipkin Server

Lezione conclusiva: si mette **tutto insieme in un unico file**, quattro container.

Cosa cambia rispetto al file della lezione 8:

1. Si aggiunge il container di Zipkin, che prende l'immagine pubblica:

```
zipkin-server:
  image: openzipkin/zipkin:2.23
  mem_limit: 300m
  ports:
    - "9411:9411"
  networks:
    - prova-network
```

2. Si sale di versione sulle immagini: **hello-world-2** va alla **0.0.3-SNAPSHOT** (la versione configurata per lavorare con Zipkin) e **il gateway** alla **0.0.2-SNAPSHOT** (idem).
3. A **entrambi** i servizi si aggiunge, tra le variabili d'ambiente, l'indirizzo di Zipkin — con la stessa identica logica di naming-server, cioè **usando il nome del container come hostname**:

```
environment:
```

EUREKA.CLIENT.SERVICEURL.DEFAULTZONE: <http://naming-server:8761/eureka>

SPRING.ZIPKIN.BASEURL: <http://zipkin-server:9411>

Avvio unico:

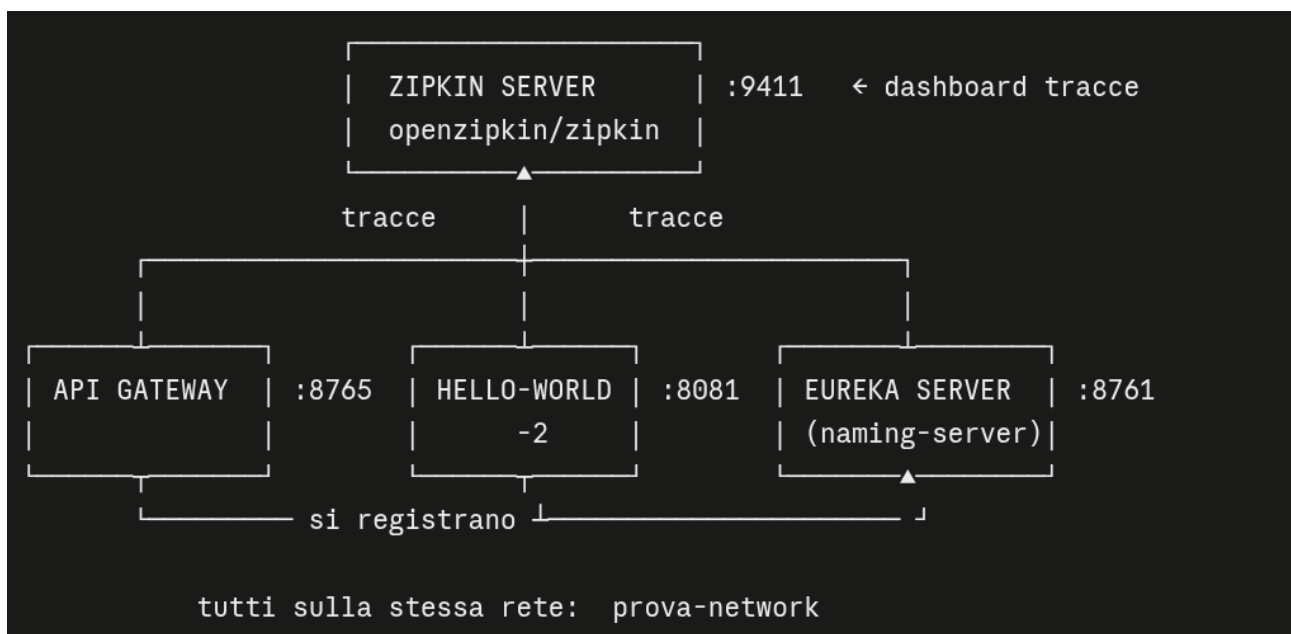
`docker-compose up`

Partono **quattro container**: l'applicazione Spring Boot con il servizio REST, l'Eureka server, l'API gateway e lo Zipkin server. Verifiche: `localhost:8081/esempio` risponde, `localhost:9411` mostra le tracce, `localhost:8761` mostra le istanze registrate compreso il gateway.

La chiusura, importante. Docker Compose serve a comporre **pochi container**. Nel mondo reale, con applicazioni grandi, per **orchestrare** tutto si usa **Kubernetes** — che il docente rimanda a un video successivo.

Quando d'insieme

L'architettura finale del corso:



I comandi da avere in testa:

Comando	Cosa fa
<code>docker run </code>	crea e avvia un container (scarica l'immagine se manca)
<code>docker run -p 8081:8080 </code>	mappa portaEsterna:portaInterna — senza questo non lo raggiungi
<code>docker run -d </code>	detach: gira in background, indipendente dal prompt
<code>docker images</code>	immagini presenti in locale
<code>docker image rm <ID></code>	rimuove un'immagine

Comando	Cosa fa
<code>docker container ls</code>	container avviati
<code>docker container stop / kill / pause / unpause <ID></code>	gestione del ciclo di vita
<code>docker logs <ID> / docker logs -f <ID></code>	log / log in tail
<code>docker push :<tag></code>	pubblica l'immagine sul registry
<code>mvn spring-boot:build-image</code>	crea l'immagine da Spring Boot senza Dockerfile
<code>docker-compose up</code>	avvia tutti i servizi del file YAML nella cartella corrente

E le tre chiavi YAML che risolvono l'80% dei problemi: **ports** (altrimenti il container non è raggiungibile), **networks** (stessa rete per tutti), **environment** (per sovrascrivere gli indirizzi usando **il nome del servizio come hostname**, mai `localhost`).